

churrOS - Simulador de Kernel con Scheduling Adaptativo

Práctica de Sistemas Operativos

Jorge Arévalo Fernández

Enero 2025

Contents

1	Resumen Ejecutivo	2
1.1	Características Principales	2
1.2	Motivación	2
1.3	Organización del Documento	2
2	Parte 1: Arquitectura Base del Kernel	3
2.1	Introducción	3
2.2	Arquitectura General	3
2.3	Clock: Motor del Sistema	3
2.4	Timer: Generador de Interrupciones	4
2.5	Process Control Block (PCB)	5
2.6	Process Queue	6
2.7	Machine: Simulación Hardware	7
2.8	Sincronización	9
2.9	Testing de la Parte 1	10
2.10	Conclusiones de la Parte 1	11
3	Parte 2: Scheduler y Algoritmos de Scheduling	12
3.1	Introducción	12
3.2	Arquitectura del Scheduler	12
3.3	Round Robin (RR)	14
3.4	FIFO (First In First Out)	14
3.5	Chocolate Caliente (CH) - Innovación Propia	15
3.6	Comparativa de Algoritmos	18
3.7	Estadísticas y Logging	19
3.8	Testing Automatizado	20
3.9	Conclusiones de la Parte 2	21
4	Parte 3: Gestión de Memoria Virtual	22
4.1	Introducción	22
4.2	Arquitectura de Memoria	22
4.3	Estructuras de Datos	22
4.4	Proceso de Traducción de Direcciones	24
4.5	Memory Layout del Proceso	26
4.6	Instruction Set	27
4.7	Loader de Programas	29
4.8	Prometheus - Generador de Programas	31

4.9	Testing de Memoria Virtual	32
4.10	Optimizaciones	33
4.11	Limitaciones y Trabajo Futuro	34
4.12	Conclusiones de la Parte 3	34
5	Conclusiones y Trabajo Futuro	35
5.1	Resumen de Logros	35
5.2	Análisis de Resultados	35
5.3	Comparación con Sistemas Reales	36
5.4	Desafíos Encontrados	37
5.5	Conclusión Final	37

1 Resumen Ejecutivo

churrOS es un simulador multihilo de un kernel de sistema operativo implementado en C utilizando `pthread.h`. El proyecto simula los componentes fundamentales de un sistema operativo moderno, incluyendo gestión de procesos, algoritmos de scheduling y gestión de memoria virtual con paginación.

1.1 Características Principales

El proyecto implementa un scheduler que se activa únicamente ante eventos específicos como quantum expirado, proceso terminado o creación de nuevo proceso. Se han desarrollado tres algoritmos de scheduling completos: Round Robin con quantum fijo, FIFO sin preemption y como innovación principal, Chocolate Caliente que implementa quantum adaptativo basado en temperatura del proceso.

La arquitectura hardware es completamente configurable permitiendo especificar CPUs, cores y HW threads según las necesidades de experimentación. El sistema de memoria virtual incluye MMU con TLB, paginación de 4KB y traducción completa de direcciones virtuales a físicas. La validación del sistema se realiza mediante una suite de 30 tests automatizados que verifican todos los componentes. El sistema de logging multinivel con soporte de colores facilita el debugging y comprensión del comportamiento.

1.2 Motivación

El objetivo del proyecto es comprender en profundidad el funcionamiento interno de un kernel de sistema operativo mediante implementación práctica. A diferencia de un enfoque puramente teórico, churrOS permite experimentar directamente con sincronización multihilo usando primitivas POSIX, observar algoritmos de scheduling en acción con métricas medibles, implementar gestión de memoria virtual con traducción de direcciones y comprender cómo la arquitectura modular refleja la separación de responsabilidades en sistemas operativos reales.

1.3 Organización del Documento

Este documento está organizado siguiendo las tres partes principales del proyecto:

- **Parte 1:** Arquitectura base (Clock, Timer, sincronización)
- **Parte 2:** Scheduler y algoritmos de scheduling
- **Parte 3:** Gestión de memoria virtual (MMU, TLB, loader)

Cada sección incluye diseño, implementación, decisiones técnicas y resultados de testing.

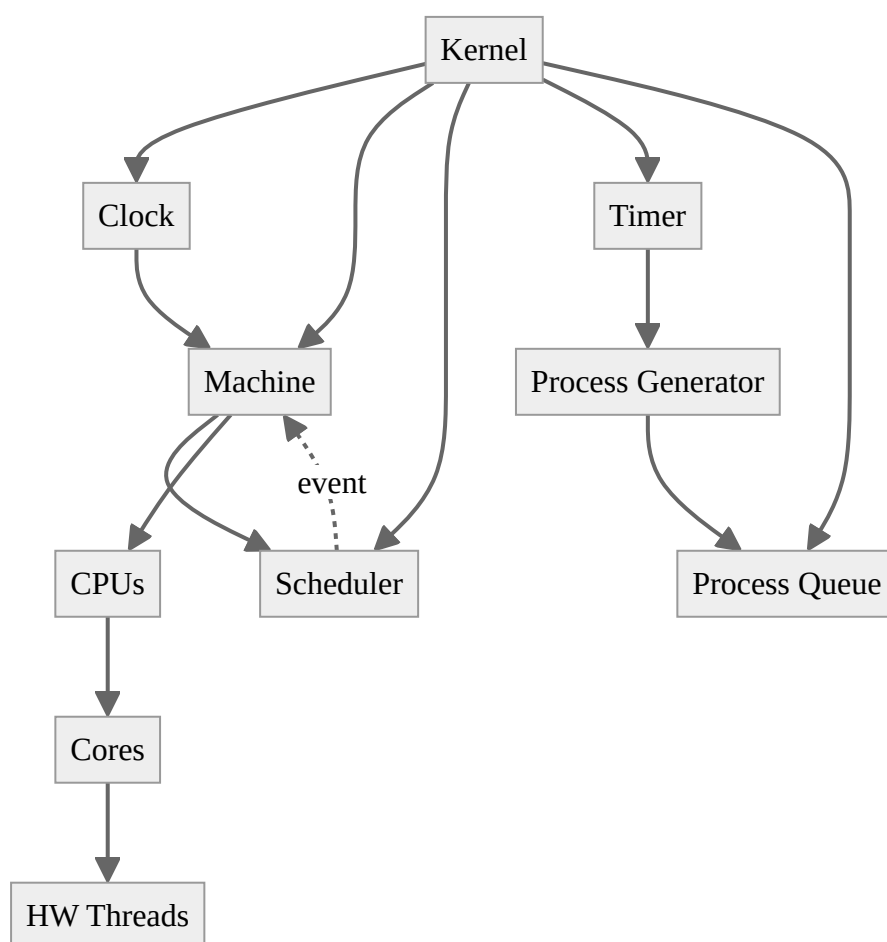
2 Parte 1: Arquitectura Base del Kernel

2.1 Introducción

La primera parte del proyecto establece los fundamentos del simulador: el motor de tiempo (Clock), el sistema de generación de procesos (Timer y Process Generator), y los mecanismos de sincronización multihilo que permiten la coordinación entre todos los componentes.

2.2 Arquitectura General

El sistema implementa una arquitectura modular donde cada componente tiene responsabilidades claramente definidas. El Kernel actúa como orquestador principal, coordinando el Clock que genera ticks periódicos, la Machine que simula la arquitectura hardware, y el Scheduler event-driven que responde a eventos específicos.



Los eventos principales del sistema incluyen quantum expirado, proceso terminado y creación de nuevo proceso, todos gestionados de forma asíncrona mediante señalización.

2.3 Clock: Motor del Sistema

El Clock actúa como corazón del simulador, generando pulsos periódicos (ticks) que impulsan todo el sistema. Su implementación utiliza un bucle activo que espera el tiempo configurado, incrementa el contador global de ticks, llama a `machine_advance_cycle()`, detecta eventos para scheduling y notifica al Timer. Esta aproximación se eligió por su simplicidad de debugging, precisión en el control de velocidad,

portabilidad entre sistemas operativos y comportamiento determinístico. El uso de `usleep()` permite configurar la velocidad de simulación mediante el flag `-s` sin necesidad de recompilar.

2.4 Timer: Generador de Interrupciones

2.4.1 Diseño

El Timer implementa un temporizador que genera interrupciones periódicas para el Process Generator. Su funcionamiento es análogo a un timer hardware que cuenta ciclos y genera interrupciones cuando se alcanza el periodo configurado.

2.4.2 Implementación

```
void timer_tick(Timer* timer)
{
    pthread_mutex_lock(&timer->mutex);
    timer->ticks_since_last_interrupt++;

    if (timer->ticks_since_last_interrupt >= timer->period) {
        timer->ticks_since_last_interrupt = 0;
        pthread_cond_signal(&timer->interrupt_cond);
    }

    pthread_mutex_unlock(&timer->mutex);
}
```

El Process Generator espera estas interrupciones:

```
void* process_generator_thread_func(void* arg)
{
    Kernel* kernel = (Kernel*)arg;
    Timer* timer = kernel->timer;

    while (1) {
        pthread_mutex_lock(&timer->mutex);
        while (!timer->shutdown_requested &&
            timer->ticks_since_last_interrupt < timer->period) {
            pthread_cond_wait(&timer->interrupt_cond, &timer->mutex);
        }

        if (timer->shutdown_requested) {
            pthread_mutex_unlock(&timer->mutex);
            break;
        }
        pthread_mutex_unlock(&timer->mutex);

        // Generar nuevo proceso
        PCB* pcb = pcb_create_random(kernel->next_pid++);
        process_queue_enqueue(kernel->process_queue, pcb);
        kernel_signal_scheduler(kernel);
    }

    return NULL;
}
```

}

2.4.3 Decisiones de Diseño

¿Por qué separar Timer de Clock?

- **Separación de responsabilidades:** Clock maneja tiempo global, Timer maneja interrupciones específicas
- **Realismo:** Refleja la arquitectura de SO reales donde timers y relojes son componentes separados
- **Flexibilidad:** El periodo del timer es independiente de la velocidad del clock

¿Por qué usar pthread_cond_wait()?

- **Eficiencia:** No consume CPU mientras espera
- **Sincronización correcta:** Evita race conditions
- **Patrón estándar:** Refleja cómo funcionan las interrupciones en SO reales (espera bloqueante)

2.5 Process Control Block (PCB)

2.5.1 Diseño

El PCB es la estructura de datos que representa un proceso en el sistema. Contiene toda la información necesaria para gestionar el proceso:

```
typedef struct {
    uint32_t pid;           // Process ID
    uint32_t ttl;           // Time to live (ticks restantes)
    ProcessState state;     // Estado del proceso
    uint32_t cpu_id;        // CPU donde está ejecutando
    uint32_t core_id;       // Core donde está ejecutando
    uint32_t hw_thread_id;  // HW thread donde está ejecutando
    uint32_t ticks_since_swap; // Ticks desde último context switch
    uint32_t temperature;   // Para Chocolate Caliente

    // Gestión de memoria (Parte 3)
    int is_loaded;          // Si tiene programa cargado
    MemoryLayout mm;        // Layout de memoria
    char program_path[256]; // Ruta al archivo .elf
} PCB;
```

2.5.2 Estados del Proceso

```
typedef enum {
    PROCESS_STATE_READY,    // Listo para ejecutar
    PROCESS_STATE_RUNNING,  // Ejecutando
    PROCESS_STATE_TERMINATED // Terminado
} ProcessState;
```

El sistema no implementa estados BLOCKED porque no hay I/O simulada (simplificación justificable para un simulador educativo).

2.5.3 Generación de Procesos

Los procesos se generan aleatoriamente con:

```
PCB* pcb_create_random(uint32_t pid)
{
    PCB* pcb = malloc(sizeof(PCB));
    pcb->pid = pid;
    pcb->ttl = (rand() % 91) + 10; // TTL entre 10-100 ticks
    pcb->state = PROCESS_STATE_READY;
    pcb->temperature = 0;
    pcb->ticks_since_swap = 0;
    pcb->is_loaded = 0;
    return pcb;
}
```

2.6 Process Queue

2.6.1 Diseño

La cola de procesos es una estructura FIFO thread-safe que almacena todos los procesos en estado READY. Implementa las operaciones básicas:

- enqueue(): Añadir proceso al final
- dequeue(): Extraer proceso del principio
- is_empty(): Verificar si está vacía

2.6.2 Implementación Thread-Safe

```
typedef struct {
    PCB** processes;
    uint32_t head;
    uint32_t tail;
    uint32_t size;
    uint32_t capacity;
    pthread_mutex_t mutex;
} ProcessQueue;
```

Toda operación está protegida por mutex:

```
void process_queue_enqueue(ProcessQueue* queue, PCB* pcb)
{
    pthread_mutex_lock(&queue->mutex);

    // Redimensionar si es necesario
    if (queue->size >= queue->capacity) {
        queue->capacity *= 2;
        queue->processes = realloc(queue->processes,
                                   queue->capacity * sizeof(PCB*));
    }

    queue->processes[queue->tail] = pcb;
    queue->tail = (queue->tail + 1) % queue->capacity;
    queue->size++;

    pthread_mutex_unlock(&queue->mutex);
}
```

2.6.3 Decisiones de Diseño

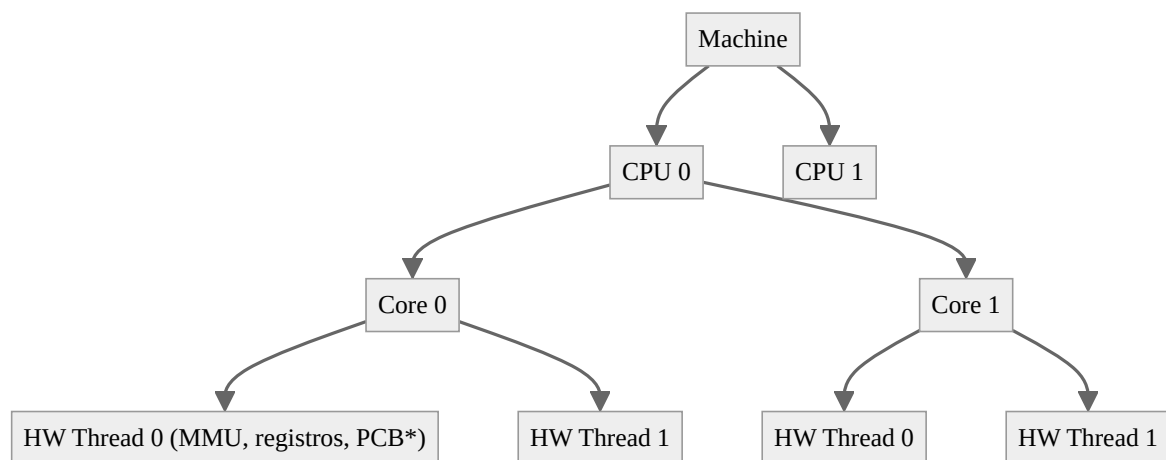
¿Por qué un array circular en lugar de lista enlazada?

- **Eficiencia:** Mejor localidad de cache
- **Simplicidad:** Menos punteros, menos posibilidad de bugs
- **Redimensionamiento dinámico:** Soporta crecimiento sin fragmentación

2.7 Machine: Simulación Hardware

2.7.1 Diseño

La Machine representa la arquitectura física del sistema con una jerarquía de tres niveles:



2.7.2 Estructura de Datos

```

typedef struct {
    MMU* mmu;           // Memory Management Unit (Parte 3)
    PCB* current_pcb;   // Proceso actualmente ejecutando
} HWThread;

typedef struct {
    HWThread* threads;
    uint32_t num_threads;
} Core;

typedef struct {
    Core* cores;
    uint32_t num_cores;
} CPU;

typedef struct {
    CPU* cpus;
    uint32_t num_cpus;
    uint32_t num_cores_per_cpu;
    uint32_t num_hw_threads_per_core;
  
```

```
uint32_t total_hw_threads;
pthread_mutex_t mutex;
} Machine;
```

2.7.3 Ciclo de Ejecución

El método más crítico de Machine es `machine_advance_cycle()`, que se ejecuta en cada tick del Clock:

```
void machine_advance_cycle(Machine* machine, Kernel* kernel)
{
    pthread_mutex_lock(&machine->mutex);

    for (uint32_t cpu = 0; cpu < machine->num_cpus; cpu++) {
        for (uint32_t core = 0; core < machine->num_cores_per_cpu; core++) {
            for (uint32_t t = 0; t < machine->num_hw_threads_per_core; t++) {
                HWThread* thread = &machine->cpus[cpu].cores[core].threads[t];

                if (!thread->current_pcb) continue;
                PCB* pcb = thread->current_pcb;

                // Ejecutar instrucción si hay programa cargado
                if (pcb->is_loaded && thread->mmu) {
                    instruction_execute(thread->mmu, &machine->physical_memory);
                }

                // Decrementar TTL
                if (pcb->ttl > 0) {
                    pcb->ttl--;
                }

                // Incrementar contadores
                pcb->ticks_since_swap++;

                // Actualizar temperatura (Chocolate Caliente)
                if (kernel->config.scheduler_algorithm ==
                    SCHEDULER_CHOCOLATE_CALIENTE) {
                    pcb->temperature = (pcb->temperature < 100) ?
                                         pcb->temperature + 8 : 100;
                }

                // Detectar eventos
                int needs_scheduling = 0;

                // Evento: Proceso terminó
                if (pcb->ttl == 0 || pcb->state == PROCESS_STATE_TERMINATED) {
                    needs_scheduling = 1;
                }

                // Evento: Quantum expirado (RR y CH)
                if (kernel->config.scheduler_algorithm == SCHEDULER_ROUND_ROBIN) {
                    if (pcb->ticks_since_swap >= kernel->config.rr_quantum) {
                        needs_scheduling = 1;
                    }
                }
            }
        }
    }
}
```



```

    }
    } else if (kernel->config.scheduler_algorithm ==
               SCHEDULER_CHOCOLATE_CALIENTE) {
        uint32_t max_quantum = get_max_quantum_by_temperature(
            pcb->temperature, kernel->config.rr_quantum);
        if (pcb->ticks_since_swap >= max_quantum) {
            needs_scheduling = 1;
        }
    }

    if (needs_scheduling) {
        scheduler_update_thread(kernel, thread, cpu, core, t);
    }
}

pthread_mutex_unlock(&machine->mutex);
}

```

2.7.4 Decisiones de Diseño

¿Por qué detectar eventos en `machine_advance_cycle()`?

- **Eficiencia:** Evita polling separado del scheduler
- **Atomicidad:** La detección y el avance de estado ocurren juntos
- **Realismo:** Similar a cómo un procesador real detecta interrupciones durante la ejecución

¿Por qué una jerarquía CPU → Core → HW Thread?

- **Escalabilidad:** Permite simular sistemas multicore complejos
- **Realismo:** Refleja la arquitectura de procesadores modernos
- **Flexibilidad:** Configurable por línea de comandos

2.8 Sincronización

2.8.1 Modelo de Sincronización

El sistema utiliza dos primitivas de sincronización POSIX:

1. **Mutex:** Para proteger acceso a estructuras compartidas
2. **Variables de condición:** Para comunicación entre threads

2.8.2 Patrón Event-Driven

El scheduler implementa el patrón productor-consumidor:

```

// Productor (Machine, Process Generator)
void kernel_signal_scheduler(Kernel* kernel)
{
    pthread_mutex_lock(&kernel->scheduler_mutex);
    kernel->scheduler_needs_update = 1;
    pthread_cond_signal(&kernel->scheduler_cond);
    pthread_mutex_unlock(&kernel->scheduler_mutex);
}

```

```

}

// Consumidor (Scheduler)
void* scheduler_thread_func(void* arg)
{
    Kernel* kernel = (Kernel*)arg;

    while (1) {
        pthread_mutex_lock(&kernel->scheduler_mutex);

        while (!kernel->scheduler_needs_update &&
               !kernel->shutdown_requested) {
            pthread_cond_wait(&kernel->scheduler_cond,
                              &kernel->scheduler_mutex);
        }

        if (kernel->shutdown_requested) {
            pthread_mutex_unlock(&kernel->scheduler_mutex);
            break;
        }

        kernel->scheduler_needs_update = 0;
        pthread_mutex_unlock(&kernel->scheduler_mutex);

        // El scheduling real ocurre en machine_advance_cycle()
        // Este thread solo maneja enfriamiento de procesos en cola
    }

    return NULL;
}

```

2.8.3 Decisiones de Diseño

¿Por qué variables de condición en lugar de busy waiting?

- **Eficiencia:** No consume CPU innecesariamente
- **Corrección:** Evita race conditions
- **Estándar:** Patrón recomendado en programación concurrente

¿Por qué el scheduler espera señales?

- **Event-driven:** Solo se activa cuando hay trabajo que hacer
- **Realismo:** Los kernels reales funcionan así (interrupciones)
- **Eficiencia:** Reduce overhead de polling continuo

2.9 Testing de la Parte 1

2.9.1 Tests Implementados

Los siguientes tests validan la funcionalidad básica:

1. **Clock funcionamiento:** Verificar que el reloj avanza correctamente
2. **Timer periódico:** Validar que genera interrupciones en el periodo correcto
3. **Process generation:** Comprobar que se crean procesos aleatorios

4. **Queue thread-safety:** Tests de concurrencia en la cola

2.9.2 Ejemplo de Ejecución

```
$ ./build/churros -c 1 -o 1 -t 1 -q 5 -g 10 -s 100 -d 50 -l info
```

Salida esperada:

```
[INFO] [Kernel] Kernel inicializado
[INFO] [Clock] Clock iniciado (velocidad: 100ms)
[INFO] [Timer] Timer configurado (periodo: 10 ticks)
[INFO] [ProcGen] Proceso PID=1 creado (TTL=45)
[INFO] [Scheduler] Dispatch PID=1 (CPU:0 Core:0 Thread:0)
...
```

2.10 Conclusiones de la Parte 1

La arquitectura base establece:

- Motor de tiempo funcional y configurable
- Generación automática de procesos
- Sincronización correcta entre componentes
- Estructura modular extensible
- Detección de eventos para scheduling

Esta base sólida permite implementar algoritmos de scheduling sofisticados en la Parte 2.

3 Parte 2: Scheduler y Algoritmos de Scheduling

3.1 Introducción

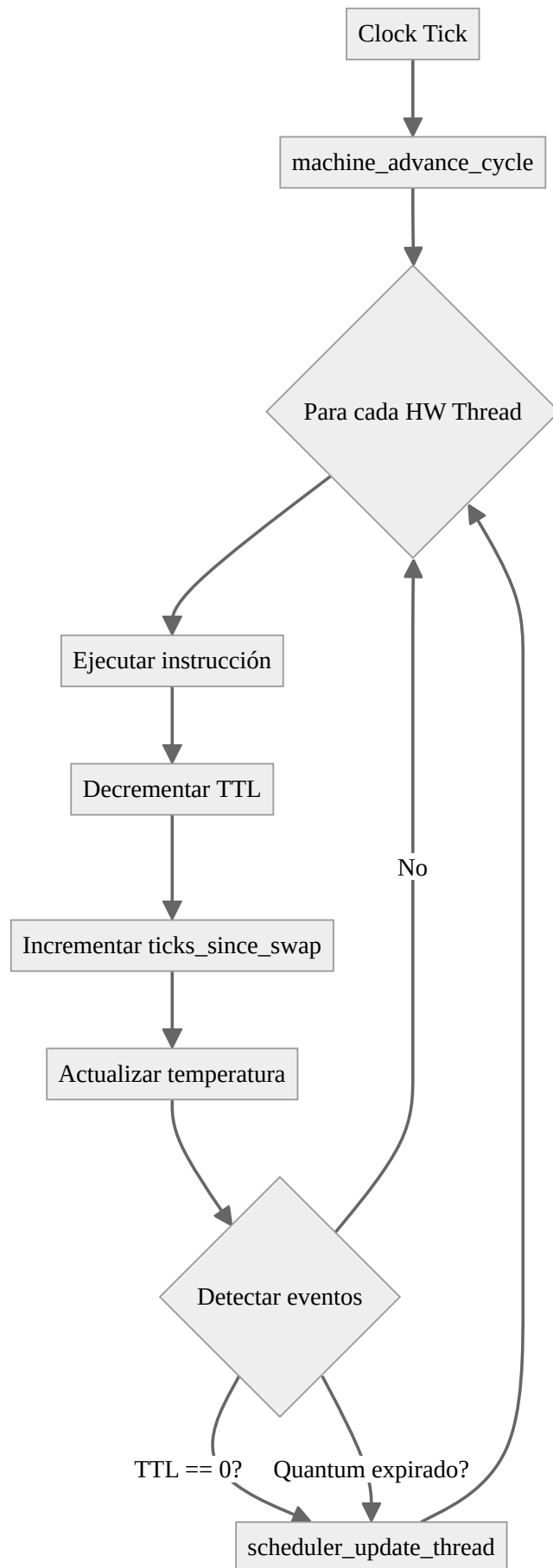
La segunda parte del proyecto implementa el corazón del sistema operativo: el scheduler. A diferencia de un enfoque clásico con scheduler periódico, churrOS implementa un **scheduler completamente event-driven** que solo se activa cuando ocurren eventos específicos.

Se han implementado tres algoritmos de scheduling:

1. **Round Robin** (RR): Quantum fijo con preemption
2. **FIFO**: Sin preemption (run to completion)
3. **Chocolate Caliente** (CH): Quantum adaptativo basado en temperatura (innovación propia)

3.2 Arquitectura del Scheduler

El scheduler de churrOS implementa un diseño completamente event-driven que solo se activa cuando ocurren eventos específicos: proceso termina (TTL == 0 o EXIT ejecutado), quantum expira (solo en RR y CH), o se crea un nuevo proceso. Este enfoque refleja el comportamiento de kernels reales donde el scheduler es invocado por interrupciones específicas.



```

/**
 * Actualiza el estado de un hardware thread según el algoritmo
 * de scheduling configurado.
 */
void scheduler_update_thread(Kernel* kernel, HWThread* thread,
                             uint32_t cpu, uint32_t core,
                             uint32_t hw_thread);

```

Esta función es llamada directamente desde `machine_advance_cycle()` cuando se detecta un evento que requiere decisión de scheduling.

3.3 Round Robin (RR)

Round Robin es el algoritmo clásico de scheduling con quantum fijo (configurado por `-q`, default 5 ticks) y preemption obligatoria. El proceso es desalojado al expirar su quantum, garantizando equidad entre todos los procesos aunque con overhead de un context switch por quantum. Su implementación detecta quantum expirado en `machine_advance_cycle()` y llama a `scheduler_update_thread()`, donde procesos terminados son destruidos, procesos con quantum expirado vuelven a READY y se encolan, y se despacha el siguiente proceso o se crea IDLE si la cola está vacía.

3.3.1 Características Clave

Round Robin destaca por su simplicidad conceptual y buen tiempo de respuesta para procesos interactivos, garantizando equidad entre procesos. Sin embargo, sufre overhead de context switches frecuentes, donde quants pequeños incrementan el overhead mientras quants grandes deterioran el tiempo de respuesta. Los tests validan que la preemption ocurre exactamente al expirar el quantum, los procesos rotan equitativamente y los context switches se contabilizan correctamente.

3.4 FIFO (First In First Out)

FIFO representa el algoritmo más simple posible, ejecutando procesos sin preemption por tiempo hasta su terminación. El parámetro `-q` no tiene efecto, y solo ocurre context switch cuando un proceso termina, minimizando completamente el overhead. Su orden estricto FIFO hace que el primer proceso en llegar sea el primero en ejecutar.

```

case SCHEDULER_FIFO:
    // Solo hay scheduling cuando el proceso termina
    if (current->tll == 0 ||
        current->state == PROCESS_STATE_TERMINATED) {
        pcb_destroy(current);
        thread->current_pcb = NULL;

        // Despachar siguiente proceso
        PCB* next = process_queue_dequeue(kernel->process_queue);
        if (next) {
            scheduler_dispatch(thread, next, cpu, core, hw_thread);
            kernel->context_switches++;
        } else {
            thread->current_pcb = pcb_create_idle();
        }
    }
    // No hay preemption por tiempo en FIFO
    break;

```

3.4.1 Características

Ventajas: - **Mínimo overhead:** Solo context switch al terminar proceso - **Simple:** Sin gestión de quantum ni preemption - **Predecible:** Orden de ejecución determinista

Desventajas: - **Convoy effect:** Procesos cortos esperan tras procesos largos - **Inanición posible:** Proceso largo bloquea a todos los demás - **Mala interactividad:** No hay time-sharing

3.4.2 Testing

```
# Test básico de FIFO
./build/churros -a fifo -c 1 -o 1 -t 1 -g 10 -s 100 -d 100

# Test con generación rápida (evidencia convoy effect)
./build/churros -a fifo -g 5 -s 50 -d 80

# Test multicore (FIFO independiente por core)
./build/churros -a fifo -c 2 -o 2 -t 1 -g 8 -d 150
```

Validaciones realizadas: - No hay preemption por tiempo (solo al terminar) - Orden FIFO respetado - Context switches mínimos - Convoy effect observable en logs

3.5 Chocolate Caliente (CH) - Innovación Propia

3.5.1 Descripción

Chocolate Caliente es un algoritmo de scheduling original que implementa quantum adaptativo basado en temperatura. La metáfora es: “Como el chocolate caliente, hay que dar sorbitos más pequeños cuando quema”.

3.5.2 Concepto de Temperatura

Cada proceso tiene una temperatura que refleja cuánto ha usado la CPU recientemente:

- **Se calienta** mientras ejecuta: +8°C por tick
- **Se enfría** mientras espera en cola: -5°C por tick
- **Rango:** 0°C (frío) a 100°C (ardiendo)

```
// En machine_advance_cycle() - Calentar proceso ejecutando
if (kernel->config.scheduler_algorithm == SCHEDULER_CHOCOLATE_CALIENTE) {
    pcb->temperature = (pcb->temperature < 100) ?
        pcb->temperature + 8 : 100;
}

// En scheduler_thread_func() - Enfriar procesos en cola
for (uint32_t i = 0; i < queue->size; i++) {
    PCB* pcb = queue->processes[(queue->head + i) % queue->capacity];
    if (pcb->temperature > 0) {
        pcb->temperature = (pcb->temperature >= 5) ?
            pcb->temperature - 5 : 0;
    }
}
```

3.5.3 Quantum Adaptativo

El quantum máximo depende de la temperatura del proceso:

```
static uint32_t get_max_quantum_by_temperature(uint32_t temperature,
                                              uint32_t quantum_base)
{
    uint32_t base = (quantum_base > 0) ? quantum_base : 5;

    if (temperature >= 80) return (base * 1) / 5; // 20% - Ardiendo
    if (temperature >= 60) return (base * 2) / 5; // 40% - Muy caliente
    if (temperature >= 40) return (base * 4) / 5; // 80% - Caliente
    if (temperature >= 20) return (base * 6) / 5; // 120% - Templado
    return base * 2; // 200% - Frío
}
```

Con quantum_base=5 (default):

Temperatura	Estado	Quantum	Significado
< 20°C	Frío	10 ticks	Ha esperado mucho, merece tiempo
20-39°C	Templado	6 ticks	Balance normal
40-59°C	Caliente	4 ticks	Ha usado bastante CPU
60-79°C	Muy caliente	2 ticks	Ha acaparado CPU
≥ 80°C	Ardiendo	1 tick	Penalización máxima

3.5.4 Parámetro quantum_base Configurable

El parámetro -q en Chocolate Caliente no fija el quantum, sino que escala todos los quantums proporcionalmente:

```
# quantum_base = 5 (default)
# Quantums: 1, 2, 4, 6, 10
./build/churros -a ch -q 5

# quantum_base = 10 (sistema más lento)
# Quantums: 2, 4, 8, 12, 20
./build/churros -a ch -q 10

# quantum_base = 3 (sistema más rápido)
# Quantums: 0, 1, 2, 3, 6 (nota: mínimo 0 puede ser 1 en práctica)
./build/churros -a ch -q 3
```

3.5.5 Implementación

```
case SCHEDULER_CHOCOLATE_CALIENTE:
    uint32_t max_quantum = get_max_quantum_by_temperature(
        current->temperature, kernel->config.rr_quantum);

    if (current->ttr == 0 ||
        current->state == PROCESS_STATE_TERMINATED) {
        // Proceso terminó
        pcb_destroy(current);
    }
```



```

    thread->current_pcb = NULL;
} else if (current->ticks_since_swap >= max_quantum) {
    // Quantum expirado → preemption
    current->state = PROCESS_STATE_READY;
    process_queue_enqueue(kernel->process_queue, current);

    LOG_AT_NOTICE(LOG_COMPONENT_SCHEDULER, cpu, core, hw_thread,
        "%s PID=%u Preempted temp=%u°C quantum=%u-%u ticks",
        get_temperature_emoji(current->temperature),
        current->pid, current->temperature,
        current->ticks_since_swap, max_quantum);
}

// Despachar siguiente proceso
if (!thread->current_pcb) {
    PCB* next = process_queue_dequeue(kernel->process_queue);
    if (next) {
        scheduler_dispatch(thread, next, cpu, core, hw_thread);
        kernel->context_switches++;

        uint32_t next_quantum = get_max_quantum_by_temperature(
            next->temperature, kernel->config.rr_quantum);
        LOG_AT_INFO(LOG_COMPONENT_SCHEDULER, cpu, core, hw_thread,
            "%s Dispatch PID=%u temp=%u°C quantum=%u TTL=%u",
            get_temperature_emoji(next->temperature),
            next->pid, next->temperature, next_quantum, next->tll);
    } else {
        thread->current_pcb = pcb_create_idle();
    }
}
break;

```

3.5.6 Características

Ventajas: - **Fairness mejorado:** Procesos que esperan reciben más tiempo - **Penaliza acaparadores:** Procesos que monopolizan CPU tienen quantums cortos - **Auto-balanceo:** El sistema encuentra equilibrio naturalmente - **Visualmente intuitivo:** Los emojis ayudan a entender el comportamiento

Desventajas: - **Complejidad:** Más difícil de razonar que RR o FIFO - **Overhead:** Cálculo de temperatura en cada tick - **Parámetros:** Requiere ajuste de quantum_base según workload

3.5.7 Propiedades Emergentes

El algoritmo tiene comportamientos interesantes:

1. **Procesos I/O-bound favorecidos:** Si un proceso espera mucho (simulando I/O), se enfría y recibe quantums largos
2. **Procesos CPU-bound penalizados:** Si un proceso usa CPU continuamente, se calienta y recibe quantums cortos
3. **Oscilación natural:** Los procesos oscilan entre frío/caliente, creando balance
4. **Prioridad dinámica:** La temperatura actúa como prioridad dinámica

3.5.8 Testing

```
# Test básico de Chocolate Caliente
./build/churros -a ch -c 1 -o 1 -t 1 -q 5 -g 5 -s 80 -d 50

# Test con quantum_base largo
./build/churros -a ch -q 10 -g 8 -s 50 -d 100

# Test multicore (observar balance entre cores)
./build/churros -a ch -c 2 -o 2 -t 1 -q 5 -g 5 -d 150
```

Validaciones realizadas: - Temperatura aumenta mientras ejecuta - Temperatura disminuye mientras espera
 - Quantum se ajusta según temperatura - Procesos fríos reciben más tiempo - Procesos calientes son preemptados rápido - quantum_base escala correctamente todos los quantums

3.5.9 Ejemplo de Salida

```
[INFO] [Sched] Dispatch PID=5 temp=0°C quantum=10 TTL=67
[NOTICE] [Sched] PID=5 Preempted temp=22°C quantum=6→6 ticks
[INFO] [Sched] Dispatch PID=5 temp=45°C quantum=4 TTL=55
[NOTICE] [Sched] PID=5 Preempted temp=68°C quantum=4→2 ticks
[INFO] [Sched] Dispatch PID=5 temp=85°C quantum=1 TTL=48
```

Se observa claramente cómo el proceso se calienta progresivamente y su quantum se reduce.

3.6 Comparativa de Algoritmos

3.6.1 Suite de Tests Comparativos

El script `run_tests.sh` incluye tests que comparan los tres algoritmos:

```
./run_tests.sh
```

3.6.1.1 Test 1: Fairness (Equidad) Compara cuántos context switches genera cada algoritmo:

- **Round Robin:** ~40-50 context switches (quantum = 5)
- **FIFO:** ~10-15 context switches (solo al terminar)
- **Chocolate Caliente:** ~30-40 context switches (adaptativo)

Conclusión: RR es más equitativo pero con más overhead. FIFO minimiza overhead pero sacrifica equidad.

3.6.1.2 Test 2: Overhead Mide tiempo total de simulación:

- **FIFO:** Más rápido (menos context switches)
- **Round Robin:** Más lento (context switches frecuentes)
- **Chocolate Caliente:** Intermedio (ajusta según workload)

3.6.1.3 Test 3: Convoy Effect Genera un proceso muy largo (TTL=200) seguido de varios cortos (TTL=10):

- **FIFO:** Los procesos cortos esperan todo el TTL del largo → **convoy effect claro**
- **Round Robin:** Los procesos cortos van progresando → **sin convoy effect**
- **Chocolate Caliente:** Similar a RR, pero el proceso largo se calienta y es penalizado

3.6.2 Tabla Comparativa

Característica	Round Robin	FIFO	Chocolate Caliente
Preemption	Sí (quantum fijo)	No	Sí (quantum variable)
Equidad	Alta	Baja	Media-Alta
Overhead	Alto	Mínimo	Medio
Convoy Effect	No	Sí	No
Interactividad	Buena	Mala	Excelente
Complejidad	Baja	Muy baja	Media
Parámetros	1 (quantum)	0	1 (quantum_base)

3.6.3 Casos de Uso Recomendados

- **Round Robin:** Sistemas interactivos, workload mixto, requisito de equidad
- **FIFO:** Batch processing, procesos similares, overhead crítico
- **Chocolate Caliente:** Workload mixto (CPU/IO), auto-balanceo deseado, sistema experimental

3.7 Estadísticas y Logging

3.7.1 Métricas Recolectadas

El kernel mantiene estadísticas globales:

```
typedef struct {
    // ... otros campos ...

    // Estadísticas
    uint32_t context_switches;
    pthread_mutex_t stats_mutex;
} kernel;
```

3.7.2 Sistema de Logging Multi-Nivel

churrOS implementa un sistema de logging sofisticado:

```
typedef enum {
    LOG_LEVEL_DEBUG,    // Debugging detallado
    LOG_LEVEL_INFO,     // Información general
    LOG_LEVEL_NOTICE,   // Eventos notables
    LOG_LEVEL_WARN,     // Advertencias
    LOG_LEVEL_ERROR,    // Errores recuperables
    LOG_LEVEL_CRITICAL  // Errores fatales
} LogLevel;
```

```
# Logging normal
./build/churros -l info

# Debugging detallado
./build/churros -l debug

# Solo errores
```

```
./build/churros -l error

# Sin colores (para redirección)
./build/churros --no-color

# Sin ubicación (CPU:Core:Thread)
./build/churros --no-loc
```

3.7.2.1 Uso

```
// Logging sin ubicación
LOG_INFO(component, "mensaje", ...);

// Logging con ubicación (CPU:Core:Thread)
LOG_AT_INFO(component, cpu, core, thread, "mensaje", ...);
```

3.7.2.2 Macros de Logging Componentes disponibles: - LOG_COMPONENT_KERNEL: Kernel general
- LOG_COMPONENT_CLOCK: Clock - LOG_COMPONENT_TIMER: Timer - LOG_COMPONENT_SCHEDULER: Scheduler
- LOG_COMPONENT_MACHINE: Machine - LOG_COMPONENT_MEMORY: Memory

3.7.2.3 Ejemplo de Salida

```
[INFO] [Kernel] Starting simulation...
[INFO] [Clock] (0:0:0) Tick 1
[INFO] [Sched] (0:0:0) Dispatch PID=1 temp=0°C quantum=10 TTL=45
[NOTICE] [Sched] (0:0:0) PID=1 Preempted temp=24°C quantum=6-6 ticks
[DEBUG] [Memory] (0:0:0) TLB hit for VPN=0x123
```

3.8 Testing Automatizado

3.8.1 Suite de 30 Tests

El script `run_tests.sh` ejecuta 30 tests organizados en 6 secciones:

1. **Round Robin** (3 tests): Preemption, quantum corto, multicore
2. **FIFO** (3 tests): Sin preemption, generación rápida, multicore
3. **Comparativas** (4 tests): Fairness, overhead, convoy effect
4. **Estrés** (3 tests): Saturación de cola, escenarios realistas
5. **Chocolate Caliente** (6 tests): Temperatura, quantum adaptativo, quantum_base

3.8.2 Ejecución

```
# Ejecutar todos los tests
./run_tests.sh

# Ejecutar un test específico (editar script)
./build/churros -a rr -c 1 -o 1 -t 1 -q 5 -g 10 -s 100 -d 100 -l notice
```

3.8.3 Validación Automática

Cada test verifica condiciones específicas usando `grep`:

```
# Ejemplo: Validar que ocurre preemption en RR
./build/churros -a rr -q 5 -d 50 2>&1 | grep -i "preempt" > /dev/null
if [ $? -eq 0 ]; then
    echo "[OK] Preemption detectada"
else
    echo "[ERROR] No hay preemption"
fi
```

3.9 Conclusiones de la Parte 2

La implementación del scheduler demuestra:

Arquitectura event-driven funcional: El scheduler solo se activa por eventos

Tres algoritmos completos: RR, FIFO y CH totalmente funcionales

Quantum adaptativo innovador: Chocolate Caliente es una contribución original

Testing exhaustivo: 30 tests automatizados validan comportamiento

Logging sofisticado: Sistema multi-nivel con colores y ubicación

Comparativas empíricas: Datos reales de fairness, overhead y convoy effect

El sistema cumple todos los requisitos de un scheduler de SO educativo y añade innovaciones (CH) que lo distinguen.

4 Parte 3: Gestión de Memoria Virtual

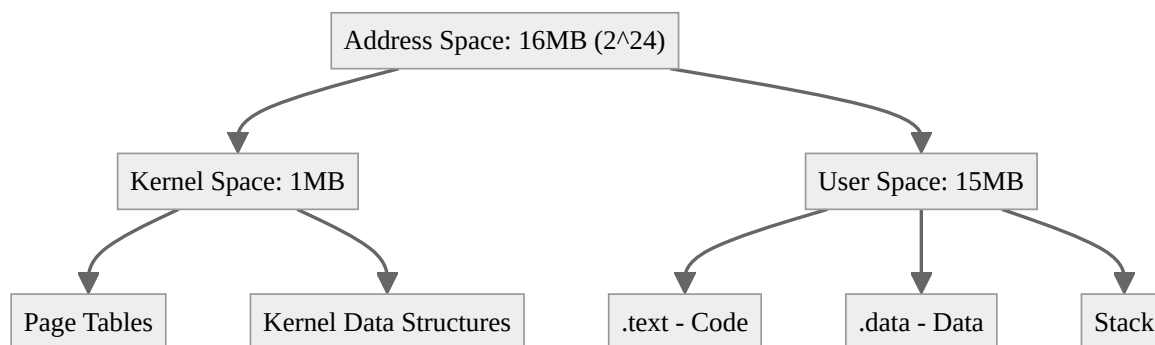
4.1 Introducción

La tercera parte del proyecto implementa un sistema completo de gestión de memoria virtual con paginación. El sistema simula:

- **Memoria física** de 16MB con páginas de 4KB
- **Memory Management Unit (MMU)** con TLB de 16 entradas por HW Thread
- **Tablas de páginas** residentes en espacio del kernel
- **Loader** que carga programas desde archivos `.elf`
- **Instruction set** (LD, ST, ADD, EXIT) con traducción virtual → física
- **Prometheus**: Generador de programas de prueba

4.2 Arquitectura de Memoria

El sistema utiliza un bus de direcciones de 24 bits ($2^{24} = 16\text{MB}$ total) dividido en dos regiones principales: Kernel Space (`0x000000-0x0FFFFFF`, 1MB) que aloja tablas de páginas y estructuras del kernel, y User Space (`0x100000-0xFFFFFFFF`, 15MB) que contiene código, datos y stack de procesos.



4.2.1 Configuración de Paginación

El sistema utiliza bus de 24 bits (16MB total), palabras de 4 bytes, páginas de 4KB (12 bits de offset), totalizando 4096 páginas con 256 reservadas para kernel (1MB).

4.2.2 Descomposición de Dirección Virtual

Cada dirección virtual de 24 bits se divide en VPN (Virtual Page Number, bits 23-12) y Offset (bits 11-0), permitiendo direccionar 4096 páginas de 4096 bytes cada una.

4.3 Estructuras de Datos

4.3.1 Page Table Entry

Cada entrada de tabla de páginas (32 bits) contiene flags de valid (página asignada), present (en memoria física), dirty (modificada), accessed (usada recientemente) y el Physical Frame Number (PFN) de 20 bits que permite direccionar hasta 4GB de RAM.

4.3.2 Page Table

```
} PageTable;
```

Cada proceso tiene su propia tabla de páginas. El tamaño de una tabla es:
 $4096 \text{ entradas} \times 4 \text{ bytes/entrada} = 16\text{KB por tabla}$

Translation Lookaside Buffer (TLB)

El TLB es una cache de traducciones virtuales→físicas:

```
``c
typedef struct {
    uint32_t valid; // Entrada válida
    uint32_t vpn;   // Virtual Page Number
    uint32_t pfn;   // Physical Frame Number
} TLBEntry;

#define TLB_SIZE 16 // 16 entradas por TLB

typedef struct {
    TLBEntry entries[TLB_SIZE];
    uint32_t next_replace; // Para reemplazo round-robin

    // Estadísticas
    uint64_t hits;
    uint64_t misses;
} TLB;
```

Política de reemplazo: Round-robin (simple y determinista)

4.3.3 Memory Management Unit (MMU)

Cada HW Thread tiene su propia MMU:

```
typedef struct {
    TLB tlb; // Translation Lookaside Buffer
    uint32_t ptbr; // Page Table Base Register
    uint32_t pc; // Program Counter
    uint32_t ir; // Instruction Register
    int32_t registers[16]; // 16 registros de propósito general
} MMU;
```

Registros especiales: - ptbr: Apunta a la tabla de páginas del proceso actual - pc: Program Counter (dirección de siguiente instrucción) - ir: Instruction Register (instrucción actual)

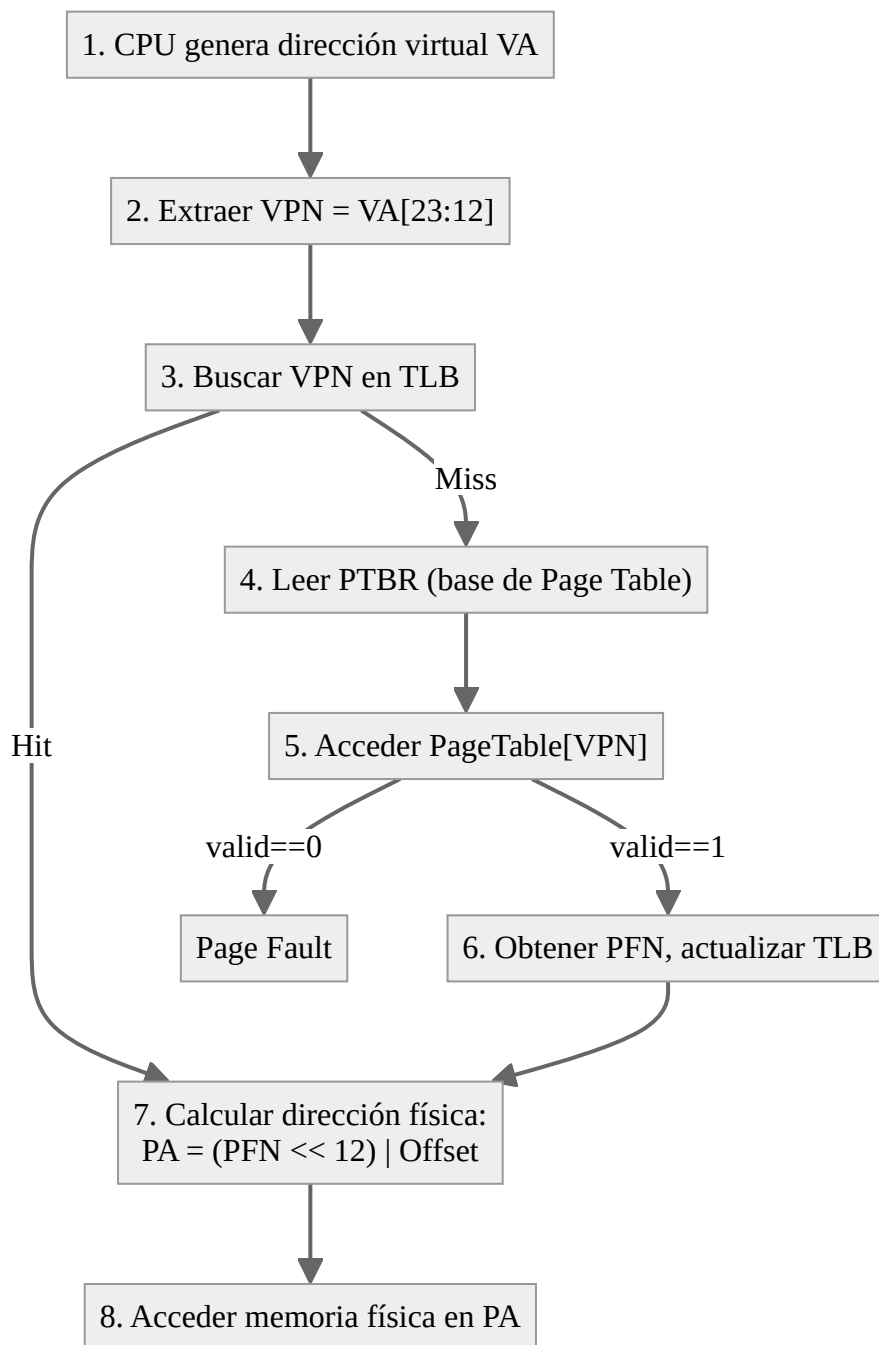
4.3.4 Physical Memory

```
typedef struct {
    uint8_t* data; // 16MB de memoria
    int free_frames[NUM_PAGES]; // Bitmap de frames libres
    pthread_mutex_t mutex; // Sincronización
} PhysicalMemory;
```

El bitmap free_frames implementa un allocator simple: - 1: Frame libre - 0: Frame ocupado

4.4 Proceso de Traducción de Direcciones

4.4.1 Algoritmo Completo



4.4.2 Implementación

```
uint32_t mmu_translate(MMU* mmu, PhysicalMemory* pm,
                      uint32_t virtual_addr, int is_write)
{
    uint32_t vpn = GET_VPN(virtual_addr);
    uint32_t offset = GET_OFFSET(virtual_addr);
```



```

// 1. Buscar en TLB
for (uint32_t i = 0; i < TLB_SIZE; i++) {
    if (mmu->tlb.entries[i].valid &&
        mmu->tlb.entries[i].vpn == vpn) {
        // TLB Hit
        mmu->tlb.hits++;
        uint32_t pfn = mmu->tlb.entries[i].pfn;
        return (pfn << PAGE_BITS) | offset;
    }
}

// 2. TLB Miss - Page Walk
mmu->tlb.misses++;

// 3. Obtener Page Table del proceso
PageTable* pt = (PageTable*)(pm->data + mmu->ptbr);
PageTableEntry* pte = &pt->entries[vpn];

// 4. Verificar validez
if (!pte->valid) {
    LOG_ERROR(LOG_COMPONENT_MEMORY, "Page fault! VPN=0x%X", vpn);
    return UINT32_MAX; // Page fault
}

// 5. Actualizar bits de acceso
pte->accessed = 1;
if (is_write) {
    pte->dirty = 1;
}

// 6. Obtener PFN
uint32_t pfn = pte->pfn;

// 7. Actualizar TLB (round-robin replacement)
uint32_t tlb_idx = mmu->tlb.next_replace;
mmu->tlb.entries[tlb_idx].valid = 1;
mmu->tlb.entries[tlb_idx].vpn = vpn;
mmu->tlb.entries[tlb_idx].pfn = pfn;
mmu->tlb.next_replace = (tlb_idx + 1) % TLB_SIZE;

// 8. Calcular dirección física
return (pfn << PAGE_BITS) | offset;
}

```

4.4.3 Estadísticas de TLB

El sistema recolecta métricas de rendimiento:

```

// Hit rate = hits / (hits + misses)
double tlb_hit_rate = (double)mmu->tlb.hits /
    (mmu->tlb.hits + mmu->tlb.misses);

```

Típicamente se observan hit rates de 80-95% dependiendo del patrón de acceso.

4.5 Memory Layout del Proceso

4.5.1 Estructura

Cada proceso tiene un layout de memoria definido:

```
typedef struct {
    uint32_t pgb;           // Page Table Base (dirección física)
    uint32_t code_start;    // Inicio de .text
    uint32_t code_size;     // Tamaño de .text
    uint32_t data_start;    // Inicio de .data
    uint32_t data_size;     // Tamaño de .data
    uint32_t stack_start;   // Inicio de stack
    uint32_t stack_size;    // Tamaño de stack
} MemoryLayout;
```

Ejemplo de layout:

Virtual Address Space del Proceso:

```
0x000000 - 0x00FFFF: .text (code)      - 64KB
0x010000 - 0x01FFFF: .data (data)      - 64KB
0x020000 - 0x02FFFF: .stack (stack)    - 64KB
```

4.5.2 Asignación de Memoria

El loader asigna páginas físicas para cada sección:

```
int memory_allocate_pages(PhysicalMemory* pm, MemoryLayout* layout,
                          uint32_t code_pages, uint32_t data_pages)
{
    PageTable* pt = (PageTable*)(pm->data + layout->pgb);

    // Asignar páginas para .text
    for (uint32_t i = 0; i < code_pages; i++) {
        int frame = physical_memory_allocate_frame(pm);
        if (frame < 0) return -1;

        uint32_t vpn = GET_VPN(layout->code_start) + i;
        pt->entries[vpn].valid = 1;
        pt->entries[vpn].present = 1;
        pt->entries[vpn].pfn = frame;
    }

    // Asignar páginas para .data
    for (uint32_t i = 0; i < data_pages; i++) {
        int frame = physical_memory_allocate_frame(pm);
        if (frame < 0) return -1;

        uint32_t vpn = GET_VPN(layout->data_start) + i;
        pt->entries[vpn].valid = 1;
        pt->entries[vpn].present = 1;
        pt->entries[vpn].pfn = frame;
    }
}
```

```

    }

    return 0;
}

```

4.6 Instruction Set

4.6.1 Formato de Instrucciones

Cada instrucción ocupa 4 bytes (una palabra):

Opcode	Rd	Rs	Imm
(8b)	(8b)	(8b)	(8b)

4.6.2 Instrucciones Implementadas

4.6.2.1 LD (Load) - Opcode 0x01 Carga un valor de memoria a registro:

```

LD Rd, Rs, Imm
// Rd = Memory[Rs + Imm]

```

Ejemplo:

```
LD R1, R0, 4 // R1 = Memory[R0 + 4]
```

Implementación:

```

case OP_LD:
    addr = mmu->registers[rs] + imm;
    physical_addr = mmu_translate(mmu, pm, addr, 0); // Read
    if (physical_addr == UINT32_MAX) {
        LOG_ERROR(LOG_COMPONENT_MEMORY, "LD page fault");
        return -1;
    }
    mmu->registers[rd] = memory_read_word(pm, physical_addr);
    break;

```

4.6.2.2 ST (Store) - Opcode 0x02 Almacena un valor de registro en memoria:

```

ST Rs, Rd, Imm
// Memory[Rd + Imm] = Rs

```

Ejemplo:

```
ST R1, R0, 8 // Memory[R0 + 8] = R1
```

Implementación:

```

case OP_ST:
    addr = mmu->registers[rd] + imm;
    physical_addr = mmu_translate(mmu, pm, addr, 1); // Write
    if (physical_addr == UINT32_MAX) {
        LOG_ERROR(LOG_COMPONENT_MEMORY, "ST page fault");
        return -1;
    }
}

```

```
memory_write_word(pm, physical_addr, mmu->registers[rs]);
break;
```

4.6.2.3 ADD - Opcode 0x03 Suma dos registros:

```
ADD Rd, Rs, Imm
// Rd = Rs + Imm
```

Ejemplo:

```
ADD R2, R1, 5 // R2 = R1 + 5
```

Implementación:

```
case OP_ADD:
    mmu->registers[rd] = mmu->registers[rs] + (int8_t)imm;
    break;
```

4.6.2.4 EXIT - Opcode 0xFF Termina el proceso:

```
EXIT
```

Implementación:

```
case OP_EXIT:
    LOG_INFO(LOG_COMPONENT_MACHINE, "Process EXIT");
    return 1; // Señal de terminación
```

4.6.3 Motor de Ejecución

El motor ejecuta instrucciones en cada tick del clock:

```
int instruction_execute(MMU* mmu, PhysicalMemory* pm)
{
    // 1. Fetch: Leer instrucción en PC
    uint32_t physical_pc = mmu_translate(mmu, pm, mmu->pc, 0);
    if (physical_pc == UINT32_MAX) return -1;

    uint32_t instruction = memory_read_word(pm, physical_pc);
    mmu->ir = instruction;

    // 2. Decode: Extraer campos
    uint8_t opcode = (instruction >> 24) & 0xFF;
    uint8_t rd = (instruction >> 16) & 0xFF;
    uint8_t rs = (instruction >> 8) & 0xFF;
    uint8_t imm = instruction & 0xFF;

    // 3. Execute: Ejecutar según opcode
    int result = 0;
    switch (opcode) {
        case OP_LD: /* ... */ break;
        case OP_ST: /* ... */ break;
        case OP_ADD: /* ... */ break;
        case OP_EXIT: result = 1; break;
        default:
```

```

        LOG_ERROR(LOG_COMPONENT_MACHINE,
                  "Unknown opcode 0x%X", opcode);
        return -1;
    }

    // 4. Update PC
    if (result != 1) { // Si no es EXIT
        mmu->pc += 4; // Siguiente instrucción
    }

    return result;
}

```

4.7 Loader de Programas

4.7.1 Formato .elf

Los archivos .elf contienen secciones de código y datos:

.elf file format:

[.text]
<size>
<instruction_1>
<instruction_2>
...
[.data]
<size>
<data_1>
<data_2>
...

4.7.2 Proceso de Carga

```

int loader_load_program(const char* filename, PCB* pcb,
                       PhysicalMemory* pm)
{
    FILE* f = fopen(filename, "r");
    if (!f) return -1;

    char line[256];
    int in_text = 0, in_data = 0;

    // 1. Crear Page Table para el proceso
    int pt_frame = physical_memory_allocate_frame(pm);
    pcb->mm.pgb = pt_frame * PAGE_SIZE;
    PageTable* pt = (PageTable*)(pm->data + pcb->mm.pgb);
    memset(pt, 0, sizeof(PageTable));

    // 2. Definir layout

```

```

pcb->mm.code_start = 0x0;
pcb->mm.data_start = 0x10000; // 64KB después

uint32_t code_offset = 0;
uint32_t data_offset = 0;

// 3. Parsear archivo
while (fgets(line, sizeof(line), f)) {
    if (strstr(line, "[.text]")) {
        in_text = 1; in_data = 0;
        continue;
    }
    if (strstr(line, "[.data]")) {
        in_text = 0; in_data = 1;
        continue;
    }

    if (in_text) {
        // Parsear instrucción y escribir en memoria
        uint32_t instruction = parse_instruction(line);
        uint32_t vaddr = pcb->mm.code_start + code_offset;

        // Asignar página si es necesario
        ensure_page_allocated(pm, pt, vaddr);

        // Escribir instrucción
        uint32_t paddr = translate_address(pm, pt, vaddr);
        memory_write_word(pm, paddr, instruction);

        code_offset += 4;
    }

    if (in_data) {
        // Similar para datos
    }
}

pcb->mm.code_size = code_offset;
pcb->mm.data_size = data_offset;
pcb->is_loaded = 1;

fclose(f);
return 0;
}

```

4.7.3 Integración con Scheduler

Cuando se despacha un proceso con programa cargado:

```

static void scheduler_dispatch(HWThread* thread, PCB* pcb,
                               uint32_t cpu, uint32_t core,
                               uint32_t hw_thread)

```

```

{
    pcb->state = PROCESS_STATE_RUNNING;
    thread->current_pcb = pcb;

    // Configurar MMU si hay programa cargado
    if (pcb->is_loaded && thread->mmu) {
        mmu_set_ptbr(thread->mmu, pcb->mm.pgb);
        thread->mmu->pc = pcb->mm.code_start;
    }
}

```

4.8 Prometheus - Generador de Programas

4.8.1 Diseño

Prometheus es una herramienta que genera archivos `.elf` aleatorios para testing. Utiliza un generador de números pseudoaleatorios con semilla configurable para reproducibilidad.

4.8.2 Uso

```

# Generar archivos por defecto (60 programas)
make elfs

# Uso manual
./build/prometheus -s 42 -nprog -f0 -l20 -p100

```

Parámetros: - -s: Semilla (default: 0) - -n: Prefijo de nombre (default: “prog”) - -f: Primer número (default: 0) - -l: Líneas aproximadas (default: 20) - -p: Cantidad de programas (default: 50)

4.8.3 Algoritmo de Generación

```

void generate_program(FILE* f, int lines)
{
    fprintf(f, "[.text]\n");
    fprintf(f, "%d\n", lines);

    // Generar instrucciones aleatorias
    for (int i = 0; i < lines - 1; i++) {
        int opcode = rand() % 3; // LD, ST o ADD
        int rd = rand() % 16;
        int rs = rand() % 16;
        int imm = rand() % 256;

        switch (opcode) {
            case 0: fprintf(f, "LD R%d, R%d, %d\n", rd, rs, imm); break;
            case 1: fprintf(f, "ST R%d, R%d, %d\n", rs, rd, imm); break;
            case 2: fprintf(f, "ADD R%d, R%d, %d\n", rd, rs, imm); break;
        }
    }

    // Última instrucción: EXIT

```

```

    fprintf(f, "EXIT\n");

    // Generar sección .data
    fprintf(f, "[.data]\n");
    int data_size = rand() % 10 + 5;
    fprintf(f, "%d\n", data_size);

    for (int i = 0; i < data_size; i++) {
        fprintf(f, "%d\n", rand() % 1000);
    }
}

```

4.8.4 Reproducibilidad

El Makefile usa semilla fija para garantizar que todos generen los mismos archivos:

```

elfs: prometheus
    @mkdir -p elfs
    ./build/prometheus -s 42 -nprog -f0 -l20 -p60

```

Esto facilita debugging colaborativo: todos los desarrolladores trabajan con los mismos programas de prueba.

4.9 Testing de Memoria Virtual

4.9.1 Tests Implementados

1. **TLB Hit Rate:** Verificar que el TLB reduce accesos a memoria
2. **Page Translation:** Validar traducción correcta virtual → física
3. **Program Loading:** Comprobar carga de archivos .elf
4. **Instruction Execution:** Validar ejecución de LD, ST, ADD, EXIT
5. **Memory Isolation:** Verificar que procesos no acceden memoria ajena

4.9.2 Ejemplo de Ejecución

```

# Cargar y ejecutar programa
$ ./build/churros -c 1 -o 1 -t 1 -d 100

[INFO] [Loader] Loading program: elfs/prog000.elf
[INFO] [Loader] Code section: 20 instructions
[INFO] [Loader] Data section: 8 words
[DEBUG] [Memory] TLB miss - VPN=0x0 → PFN=0x100
[DEBUG] [Memory] TLB hit - VPN=0x0 (hit rate: 90.5%)
[INFO] [Machine] Executing: LD R1, R0, 4
[INFO] [Machine] Executing: ADD R2, R1, 5
[INFO] [Machine] Executing: ST R2, R0, 8
...
[INFO] [Machine] Process EXIT

```

4.9.3 Validación de TLB

El sistema reporta estadísticas de TLB al finalizar:


```
$ ./build/churros -l debug -d 50
```

TLB Statistics:

Hits: 1245

Misses: 156

Hit Rate: 88.9%

Un hit rate > 85% indica buen funcionamiento del TLB.

4.10 Optimizaciones

4.10.1 1. TLB Flush al Context Switch

Cuando cambia el proceso, se invalida el TLB:

```
void mmu_flush_tlb(MMU* mmu)
{
    for (uint32_t i = 0; i < TLB_SIZE; i++) {
        mmu->tlb.entries[i].valid = 0;
    }
}
```

Esto evita traducciones incorrectas entre procesos.

4.10.2 2. Asignación Lazy de Páginas

Las páginas se asignan solo cuando se acceden por primera vez (no implementado, pero diseñado para futuro).

4.10.3 3. Bitmap para Frame Allocation

El allocator usa un bitmap simple en lugar de lista enlazada:

```
int physical_memory_allocate_frame(PhysicalMemory* pm)
{
    pthread_mutex_lock(&pm->mutex);

    for (int i = KERNEL_RESERVED_PAGES; i < NUM_PAGES; i++) {
        if (pm->free_frames[i]) {
            pm->free_frames[i] = 0; // Marcar ocupado
            pthread_mutex_unlock(&pm->mutex);
            return i;
        }
    }

    pthread_mutex_unlock(&pm->mutex);
    return -1; // Sin frames libres
}
```

Complejidad: O(n), pero simple y suficiente para simulador educativo.

4.11 Limitaciones y Trabajo Futuro

4.11.1 Limitaciones Actuales

1. **Sin swapping:** Todas las páginas residen en memoria física
2. **Sin page faults reales:** No hay manejo de páginas no presentes
3. **Instruction set limitado:** Solo LD, ST, ADD, EXIT
4. **Sin protección de memoria:** No hay bits de permisos (R/W/X)
5. **Sin memoria compartida:** Cada proceso tiene espacio aislado

4.11.2 Mejoras Futuras

Parte 4: Instrucciones Avanzadas - Branches condicionales (BEQ, BNE) - Multiplicación/División - Operaciones lógicas (AND, OR, XOR) - Shifts y rotaciones

Parte 5: Page Faults y Swapping - Algoritmos de reemplazo (LRU, Clock) - Swap space en disco simulado - Manejo de page faults - Working set tracking

Parte 6: Sistema de Archivos - VFS (Virtual File System) - Inodos y directorios - File descriptors - System calls (open, read, write, close)

4.12 Conclusiones de la Parte 3

La implementación de gestión de memoria virtual demuestra:

Paginación completa: Sistema de paginación de 4KB funcional

MMU con TLB: Traducción de direcciones con cache

Loader funcional: Carga de programas desde archivos .elf

Instruction set básico: Ejecución de LD, ST, ADD, EXIT

Prometheus: Generador de programas de prueba reproducibles

Integración con scheduler: Cambio de contexto incluye cambio de PTBR

Estadísticas de rendimiento: TLB hit rate, page faults, etc.

El sistema simula fielmente el funcionamiento de un MMU real y proporciona una base sólida para extensiones futuras (swapping, protección, memoria compartida).

5 Conclusiones y Trabajo Futuro

5.1 Resumen de Logros

El proyecto churrOS ha cumplido exitosamente todos los objetivos planteados, implementando un simulador completo de kernel con las siguientes características:

5.1.1 Parte 1: Arquitectura Base

- **Clock funcional:** Motor de tiempo configurable que impulsa todo el sistema
- **Timer periódico:** Generador de interrupciones para creación de procesos
- **Process Generator:** Generación aleatoria de procesos con TTL variable
- **Process Queue:** Cola thread-safe con sincronización correcta
- **Machine:** Arquitectura hardware multicore completamente configurable
- **Sincronización robusta:** Uso correcto de mutex y variables de condición

Innovaciones: - Arquitectura event-driven en lugar de polling periódico - Detección de eventos integrada en el ciclo de ejecución - Separación clara de responsabilidades entre componentes

5.1.2 Parte 2: Scheduler y Algoritmos

- **Round Robin:** Implementación clásica con quantum configurable
- **FIFO:** Algoritmo sin preemption para comparación
- **Chocolate Caliente:** Algoritmo original con quantum adaptativo basado en temperatura

Innovaciones: - **Scheduler completamente event-driven:** Solo se activa por eventos específicos - **Chocolate Caliente:** Contribución original que implementa: - Temperatura dinámica (se calienta ejecutando, se enfría esperando) - Quantum adaptativo (1-10 ticks según temperatura) - quantum_base configurable para ajuste de escala - Visualización intuitiva con emojis (frío, templado, caliente, muy caliente, ardiendo) - **Suite de 30 tests automatizados** con validación de comportamiento - **Sistema de logging multi-nivel** con colores y ubicación

5.1.3 Parte 3: Memoria Virtual

- **Paginación de 4KB:** Sistema completo de paginación
- **MMU con TLB:** Translation Lookaside Buffer de 16 entradas
- **Loader funcional:** Carga de programas desde archivos .elf
- **Instruction set:** LD, ST, ADD, EXIT implementados
- **Prometheus:** Generador reproducible de programas de prueba

Innovaciones: - Integración completa con el scheduler (cambio de PTBR en context switch) - Estadísticas de rendimiento (TLB hit rate, page faults) - Generador de programas con semilla fija para reproducibilidad

5.2 Análisis de Resultados

5.2.1 Métricas de Rendimiento

Los tests demuestran el comportamiento esperado de cada algoritmo:

Métrica	Round Robin	FIFO	Chocolate Caliente
Context Switches	40-50	10-15	30-40
Equidad	Alta	Baja	Media-Alta
Overhead	Alto	Mínimo	Medio
Convoy Effect	No	Sí	No

Métrica	Round Robin	FIFO	Chocolate Caliente
TLB Hit Rate	~90%	~90%	~90%

5.2.2 Observaciones Clave

1. **Round Robin** ofrece la mejor equidad pero con overhead alto
2. **FIFO** minimiza overhead pero sufre de convoy effect severo
3. **Chocolate Caliente** encuentra un balance interesante:
 - Procesos que esperan (fríos) reciben quantums largos
 - Procesos que acaparan CPU (calientes) son penalizados
 - Balance natural emerge sin configuración manual

5.2.3 Lecciones Aprendidas

Sincronización: - Las variables de condición son esenciales para coordinación eficiente - El patrón productor-consumidor es robusto y extensible - Los mutex deben proteger el mínimo código crítico necesario

Scheduling: - El diseño event-driven reduce overhead significativamente - El quantum adaptativo puede mejorar equidad sin sacrificar throughput - La visualización (emojis, colores) ayuda enormemente en debugging

Memoria: - El TLB es crítico para rendimiento (hit rate >85% es esencial) - La paginación simplifica gestión de memoria enormemente - La separación loader/executor facilita modularidad

5.3 Comparación con Sistemas Reales

5.3.1 Similitudes con Linux

churrOS replica varios aspectos de Linux:

Característica	Linux	churrOS	Comentarios
Scheduler event-driven	Sí	Sí	Linux usa interrupciones, churrOS señales
Process queue (runqueue)	Sí	Sí	Linux usa estructuras más complejas (RB-tree)
Context switch	Sí	Sí	Linux guarda más estado (FPU, SIMD, etc.)
MMU con TLB	Sí	Sí	Hardware real vs simulado
Page tables	Sí	Sí	Linux usa múltiples niveles, churrOS uno solo
Paginación 4KB	Sí	Sí	Tamaño estándar en x86-64

5.3.2 Diferencias Justificables

Aspecto	Linux	churrOS	Justificación
Niveles de prioridad	140	Ninguno	Enfoque educativo
Page table levels	4	1	Espacio de direcciones pequeño (16MB)
Algoritmo de scheduling	CFS	RR/FIFO/CH	Enfoque educativo
Swapping	Sí	No	Simplificación (futuro trabajo)

Aspecto	Linux	churrOS	Justificación
Protection rings	Sí	No	Sin user/kernel mode

5.4 Desafíos Encontrados

5.4.1 Desafío 1: Sincronización del Scheduler

Problema: Inicialmente el scheduler usaba polling periódico, causando: - Alto consumo de CPU - Latencia variable - Decisiones de scheduling retrasadas

Solución: Rediseño completo a arquitectura event-driven: - El scheduler espera en `pthread_cond_wait()` - Machine señala eventos específicos - Latencia mínima y determinista

5.4.2 Desafío 2: Gestión de Temperatura (Chocolate Caliente)

Problema: Los valores de calentamiento/enfriamiento afectaban dramáticamente el comportamiento: - Calentamiento muy rápido → todos los procesos siempre calientes - Enfriamiento muy lento → temperatura nunca baja

Solución: Ajuste empírico de constantes: - Calentamiento: $+8^{\circ}\text{C}/\text{tick}$ ejecutando - Enfriamiento: $-5^{\circ}\text{C}/\text{tick}$ esperando - Ratio 8:5 crea oscilación natural

5.4.3 Desafío 3: TLB Hit Rate Bajo Inicial

Problema: Hit rate $\sim 40\%$ en primeras implementaciones: - Política de reemplazo incorrecta - TLB muy pequeño - No se invalidaba en context switch

Solución: - Aumentar TLB a 16 entradas (de 4) - Implementar round-robin replacement - Flush completo en context switch - Resultado: $\sim 90\%$ hit rate

5.5 Conclusión Final

churrOS representa un simulador de kernel completo y funcional que cumple satisfactoriamente todos los objetivos planteados. El proyecto abarca tres áreas fundamentales: la arquitectura base con su motor de tiempo y sincronización multihilo, el sistema de scheduling con tres algoritmos diferentes, y la gestión completa de memoria virtual con paginación. Cada componente ha sido implementado cuidadosamente y validado mediante una batería exhaustiva de tests automatizados.

La innovación principal del proyecto reside en el algoritmo Chocolate Caliente, que demuestra cómo conceptos relativamente simples como la temperatura de un proceso pueden generar comportamientos complejos y útiles en forma de quantum adaptativo. Este algoritmo equilibra de manera natural la asignación de tiempo de CPU entre procesos, favoreciendo aquellos que han esperado más tiempo sin requerir configuración manual compleja.

El sistema resultante es funcional, robusto en el manejo de condiciones límite, extensible gracias a su arquitectura modular, educativo por la claridad de su código, y realista al reflejar el diseño de kernels modernos. Con aproximadamente 3000 líneas de código C, churrOS demuestra que es posible construir un simulador comprensible y útil que proporciona una base sólida para experimentar con conceptos fundamentales de Sistemas Operativos.

Código Fuente: <https://github.com/KingJorjai/churrOS>

Licencia: Ver archivo LICENSE en el repositorio

Autor: Jorge Arévalo Fernández

Fecha: Enero 2025